

Reviewer Pack — Minimal Rank of Tropicalized Affine Schemes vs Monotone Circ...

Entry 1b946aefd116 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 18:56:27 UTC

Minimal Rank of Tropicalized Affine Schemes vs Monotone Circuit Depth for k-CLIQUE

Entry ID: 1b946aefd116 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 18:56:27 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry over Affine Algebraic Geometry **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

[‘The minimal rank of a tropicalization of an affine scheme encoding the k-CLIQUE problem is $\Theta(n^{\{1/4\}})$ ’, ‘This implies that monotone circuits for k-CLIQUE require size $n^{\Omega(k\{1/4\})}$.’, ‘For all instances with property P, property Q holds.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a way to encode combinatorial problems into algebraic structures.’, ‘If the rank of these structures is low for k-CLIQUE, it suggests that the problem can be solved by monotone circuits with small depth.’, ‘This conjecture bridges tropical geometry and circuit complexity, potentially leading to new lower bounds.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0cba31a449603468

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average rank of tropicalized affine schemes across 30 random seeds is within $\Theta(n^{\{1/4\}})$ with a standard deviation not exceeding $0.1n^{\{1/4\}}$, where n is the number of vertices in the k-CLIQUE instance.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "affine algebraic geometry" AND "k-CLIQUE problem" - "monotone circuit complexity" AND " $\Theta(n^{1/4})$ " - "property P" AND "property Q" AND ("tropicalization" OR "minimal rank") AND "complexity theory"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = Fraction(A[i][i])
            for j in range(n):
                A[i][j] /= factor
            for j in range(n):
                if j != i:
                    factor = Fraction(A[j][i])
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def rank(matrix):
        n = len(matrix)
        matrix = [row[:] for row in matrix]
        gaussian_elimination(matrix)
        rank = 0
        for i in range(n):
            if any(matrix[i]):
```

```

        rank += 1
    return rank

def generate_k_clique_instance(n, k):
    vertices = list(range(n))
    edges = []
    for i in range(k):
        for j in range(i+1, k):
            edges.append((vertices[i], vertices[j]))
    return vertices, edges

n = random.randint(5, 40)
vertices, edges = generate_k_clique_instance(n, n // 2)

# Construct the affine scheme
A = [[0] * (n + 1) for _ in range(n + 1)]
for i in range(n):
    A[i][i] = Fraction(1)
    A[n][i] = Fraction(-1)
for u, v in edges:
    A[u][v] = Fraction(1)
    A[v][u] = Fraction(1)

# Tropicalize the affine scheme
tropicalized_A = [[max(A[i][j], 0) for j in range(n + 1)] for i in range(n + 1)]

# Compute the rank of the tropicalized affine scheme
rank_T = rank(tropicalized_A)

return {
    "metric_name": "Tropicalized Rank",
    "metric_value": rank_T,
    "instances_tested": 1,
    "conjecture_holds": True if n**0.25 <= rank_T <= n**0.25 else False,
    "counterexample": "" if n**0.25 <= rank_T <= n**0.25 else f"n={n}, rank={rank_T}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_rank = sum(res["metric_value"] for res in results if "metric_value" in res)
    mean_rank = Fraction(total_rank, len(results))
    std_rank = math.sqrt(sum((res["metric_value"] - mean_rank)**2 for res in results) / len(results))

    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, res in enumerate(results) if not res["conjecture"]), None)
    counterexample = results[first_failing_seed]["counterexample"]
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5689326.py", line 90, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5689326.py", line 73, in run_trial
    rank_T = rank(tropicalized_A)
              ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5689326.py", line 42, in rank
    gaussian_elimination(matrix)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5689326.py", line 31, in gaussian_elimination
    A[i][j] /= factor
  File "/usr/lib/python3.12/fractions.py", line 615, in forward
    return monomorphic_operator(a, b)
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 763, in _div
    raise ZeroDivisionError('Fraction(%s, 0)' % db)
ZeroDivisionError: Fraction(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the division by zero error in the code to allow for the completion of the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16165
2	preregistration	ollama_remote	glm4:latest	0	0	9663
3	novelty	ollama_remote	glm4:latest	0	0	8767
4	novelty	ollama_remote	glm4:latest	0	0	8708
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15421
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12922
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15416
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10641
9	judge	ollama_remote	glm4:latest	0	0	12428

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110132 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1b946aefd116.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1b946aefd116.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1b946aefd116.tar.gz` (if generated)